

Memo — Constrained Document Layout Detection & Reasoning API

1. Domain, dataset, and why

I chose document layout analysis — detecting structural regions (tables, headings, captions, formulas) on a document page — over more common choices like traffic or PPE detection for three reasons. First, all candidate classes are non-COCO, so no pretrained checkpoint can score without real fine-tuning; PPE/traffic domains lean on COCO's person/car classes for a chunk of their apparent performance. Second, it produces a genuinely architectural "insufficient information" case for Part B (below), not a tuned confidence threshold. Third, it's adjacent to what Rapid Acceleration Partners builds — RAPFlow's stated purpose is automated extraction from unstructured documents, and layout detection is the perception stage underneath that.

Dataset: pierreguillou/DocLayNet-base (Hugging Face), a 10% subset of IBM Research's DocLayNet — 8,057 pages, expert-annotated, CDLA-Permissive-1.0 license, spanning 6 document categories (financial reports, scientific articles, laws/regulations, tenders, manuals, patents). I used this subset over the full 80k-page release specifically for the time budget: the full release is ~30 GB and takes ~45 minutes just to download, which was not defensible against a one-day window. Same annotation quality, ~8x smaller.

One real data hazard: the dataset's `bboxes_block` field repeats each region's box once per text line it contains, so a naive read of a 40-region page produces ~1,100 duplicate annotations. `scripts/prepare_dataset.py` deduplicates on (`box`, `category`) before writing YOLO labels and reports the dedupe ratio per split.

2. Split strategy

I used the dataset author's published train/val/test split (6,910/648/499) rather than re-stratifying myself. This is the reproducible choice — comparable to published DocLayNet baselines — but it means inheriting whatever leakage that split has. I measured it rather than assuming: **0% of test pages share a source PDF with the training set** (see `reports/metrics.json:split_leakage`). The split is clean.

3. Evaluation and what it does / doesn't tell you

Test-set results (499 pages, RT-DETR-L, 15 epochs, Tesla T4, 2.27h): **mAP50 = 0.622, mAP50-95 = 0.412, precision = 0.706, recall = 0.605.**

I report more than one number because a single aggregate mAP cannot answer what I actually need to know. Per-document-category mAP50 ranges from 0.495 (`laws_and_regulations`) to 0.745 (`scientific_articles`) — an honest signal about which document styles generalize, not just an aggregate. I also measured RT-DETR's query-budget saturation directly rather than letting it hide inside a recall number: **0% of test pages exceeded the 300-query budget** (max regions on any page: 95), so saturation is not a factor in these results.

What mAP does not tell you: it measures box localization and classification, nothing about reading order, and nothing about whether a detected Table's internal structure is usable downstream. A model can score well here and still be useless for extracting an actual table's rows.

4. Five failure cases

1. Footnote — near-total recall collapse (recall 0.048, mAP50 0.182). I predicted before training that small/thin classes would suffer at 640px input. Only half right: Footnote has just 47 training instances across 18 images — rarity, not size, is the dominant factor (see #2's contrast).

2. Page-footer almost failed the same way, but didn't (mAP50 0.843). Equally thin, similarly positioned at the page margin, but 408 instances across 381 images (~9x more than Footnote) and a highly consistent position. This is the most useful negative result in the run: it revised my hypothesis from "small objects fail" to "small *and rare* objects fail; small-but-frequent-and-positionally-consistent ones are fine."

3. Dense repeated-entry layouts produce duplicate, jittery boxes

(`reports/failures/01_test_000035_err120.png`, financial_reports, the worst page overall: 38 missed, 82 false positives). This page is a bank's organizational directory — ~25 near-identical repeated Section-header/Text entry pairs stacked in two columns. Ground truth has one clean box per entry; the prediction shows heavily overlapping, offset duplicate boxes at scattered confidence (0.25–0.85). At 640px, the model appears unable to cleanly separate many visually near-identical, tightly packed instances — a genuine limitation of a fixed-query architecture on repetitive layouts, not a training bug.

4. Composite Picture regions get fragmented into overlapping sub-boxes

(`reports/failures/04_test_000425_err61.png`, patents: 0 missed, 0 misclassified, 61 false positives — the cleanest signature in the run). Each ground-truth Picture is one chemical structure diagram; the model predicts 2–3 overlapping

Picture boxes per diagram instead of one, apparently treating the diagram's internal sub-drawing and inline formula text as separable parts. This directly explains Picture's low precision (0.416) despite reasonable recall (0.622) — it's finding pictures, just not as single coherent regions.

5. Page-header confused with Text on running headers — observed live on a random, previously-unseen legal-code test page (not from the mined failure set): a running header/citation line ("...Official Gazette of the Republic...") was labelled Text at low confidence (0.34–0.46), well below the model's confident paragraph-level Text calls (0.68–0.96). The low confidence itself is informative — the model is uncertain about exactly the region it should be classifying differently.

5. Part B — when it detects, and the honest refusal

The router (regex prefilter → Groq LLM, strict json_schema) decides needs_detection before anything else runs. The prompt tells it explicitly that the detector's vocabulary is layout-only — it knows *where* regions are, never *what text* says. A question like "what is the invoice total" sounds document-related but is content, not layout, so it routes to out_of_scope before detection ever runs.

Real, live-tested example: asked "What law is this document about?" against an actual test page, the system responded:

"I can't answer that from this image's layout: Question asks about document content, which the detector cannot read."

insufficient_information=True here is set in code from the router/ guardrail state, not from the LLM self-reporting confidence — a confident-sounding wrong answer is one bad prompt away if the LLM judges its own certainty. The deterministic guardrail (4 rules: no detections, all below a 0.50 confidence floor, too much of a count in an ambiguous 0.25–0.50 band, or query-budget saturation) is what actually decides, and it can't be talked out of triggering by fluent-sounding synthesis text. Full trace for every response is included in the API output for auditing.